

G431-MSC-Sensors_Flandin&Dugard

Generated by Doxygen 1.15.0

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

CMSIS	??
Stm32g4xx_system	??
STM32G4xx_System_Private_Includes	??
STM32G4xx_System_Private_TypesDefinitions	??
STM32G4xx_System_Private_Defines	??
STM32G4xx_System_Private_Macros	??
STM32G4xx_System_Private_Variables	??
STM32G4xx_System_Private_FunctionPrototypes	??
STM32G4xx_System_Private_Functions	??

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

CompassCalib_t		??
--------------------------------	-------	----

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

Core/Inc/ main.h	: Header for main.c file. This file contains the common defines of the application	??
Core/Inc/ stm32g4xx_hal_conf.h	HAL configuration file	??
Core/Inc/ stm32g4xx_it.h	This file contains the headers of the interrupt handlers	??
Core/Src/ main.c	: Main program body	??
Core/Src/ stm32g4xx_hal_msp.c	This file provides code for the MSP Initialization and de-Initialization codes	??
Core/Src/ stm32g4xx_it.c	Interrupt Service Routines	??
Core/Src/ syscalls.c	STM32CubeIDE Minimal System calls file	??
Core/Src/ systemem.c	STM32CubeIDE System Memory calls file	??
Core/Src/ system_stm32g4xx.c	CMSIS Cortex-M4 Device Peripheral Access Layer System Source File	??
Utils/Inc/ compass.h	En-têtes des fonctions pour la boussole électronique	??
Utils/Inc/ functions.h	En-têtes des fonctions de gestion du MPU-9250	??
Utils/Inc/ tools.h	En-têtes des fonctions de tout début de projet	??
Utils/Src/ compass.c	Création d'une boussole électronique (mono et multi capteurs)	??
Utils/Src/ functions.c	Fonctions de gestion du capteur MPU-9250 (IMU 9 axes)	??
Utils/Src/ tools.c	Fonctions servant au tout début du projet	??

Chapter 4

Topic Documentation

4.1 CMSIS

Topics

- [Stm32g4xx_system](#) ??

4.1.1 Detailed Description

4.1.2 Stm32g4xx_system

Topics

- [STM32G4xx_System_Private_Includes](#) ??
- [STM32G4xx_System_Private_TypesDefinitions](#) ??
- [STM32G4xx_System_Private_Defines](#) ??
- [STM32G4xx_System_Private_Macros](#) ??
- [STM32G4xx_System_Private_Variables](#) ??
- [STM32G4xx_System_Private_FunctionPrototypes](#) ??
- [STM32G4xx_System_Private_Functions](#) ??

4.1.2.1 Detailed Description

4.1.2.2 STM32G4xx_System_Private_Includes

Macros

- #define [HSE_VALUE](#) 24000000U
- #define [HSI_VALUE](#) 16000000U

4.1.2.2.1 Detailed Description

4.1.2.2.2 Macro Definition Documentation

4.1.2.2.2.1 HSE_VALUE

```
#define HSE_VALUE 24000000U
```

Value of the External oscillator in Hz

4.1.2.2.2.2 HSI_VALUE

```
#define HSI_VALUE 16000000U
```

Value of the Internal oscillator in Hz

4.1.2.3 STM32G4xx_System_Private_TypeDefinitions

4.1.2.4 STM32G4xx_System_Private_Defines

4.1.2.5 STM32G4xx_System_Private_Macros

4.1.2.6 STM32G4xx_System_Private_Variables

Variables

- uint32_t **SystemCoreClock** = [HSI_VALUE](#)
- const uint8_t **AHBPrescTable** [16] = {0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U, 1U, 2U, 3U, 4U, 6U, 7U, 8U, 9U}
- const uint8_t **APBPrescTable** [8] = {0U, 0U, 0U, 0U, 1U, 2U, 3U, 4U}

4.1.2.6.1 Detailed Description

4.1.2.7 STM32G4xx_System_Private_FunctionPrototypes

4.1.2.8 STM32G4xx_System_Private_Functions

Functions

- void [SystemInit](#) (void)
Setup the microcontroller system.
- void [SystemCoreClockUpdate](#) (void)
Update SystemCoreClock variable according to Clock Register Values. The SystemCoreClock variable contains the core clock (HCLK), it can be used by the user application to setup the SysTick timer or configure other parameters.

4.1.2.8.1 Detailed Description

4.1.2.8.2 Function Documentation

4.1.2.8.2.1 SystemCoreClockUpdate()

```
void SystemCoreClockUpdate (
    void )
```

Update SystemCoreClock variable according to Clock Register Values. The SystemCoreClock variable contains the core clock (HCLK), it can be used by the user application to setup the SysTick timer or configure other parameters.

Note

Each time the core clock (HCLK) changes, this function must be called to update SystemCoreClock variable value. Otherwise, any configuration based on this variable will be incorrect.

- The system frequency computed by this function is not the real frequency in the chip. It is calculated based on the predefined constant and the selected clock source:

- If SYSCLK source is HSI, SystemCoreClock will contain the [HSI_VALUE\(**\)](#)
- If SYSCLK source is HSE, SystemCoreClock will contain the [HSE_VALUE\(***\)](#)
- If SYSCLK source is PLL, SystemCoreClock will contain the [HSE_VALUE\(***\)](#) or [HSI_VALUE\(*\)](#) multiplied/divided by the PLL factors.

(**) HSI_VALUE is a constant defined in stm32g4xx_hal.h file (default value 16 MHz) but the real value may vary depending on the variations in voltage and temperature.

(***) HSE_VALUE is a constant defined in stm32g4xx_hal.h file (default value 24 MHz), user has to ensure that HSE_VALUE is same as the real frequency of the crystal used. Otherwise, this function may have wrong result.

- The result of this function could be not correct when using fractional value for HSE crystal.

Parameters

None	
------	--

Return values

None	
------	--

4.1.2.8.2.2 SystemInit()

```
void SystemInit (
    void )
```

Setup the microcontroller system.

Parameters

<i>None</i>	
-------------	--

Return values

<i>None</i>	
-------------	--

Chapter 5

Class Documentation

5.1 CompassCalib_t Struct Reference

Public Attributes

- double **offset** [3]
- double **gain** [3]

The documentation for this struct was generated from the following file:

- Utils/Inc/[compass.h](#)

Chapter 6

File Documentation

6.1 Core/Inc/main.h File Reference

: Header for [main.c](#) file. This file contains the common defines of the application.

```
#include "stm32g4xx_hal.h"
```

Macros

- `#define RCC_OSC32_OUT_Pin` GPIO_PIN_14
- `#define RCC_OSC32_OUT_GPIO_Port` GPIOC
- `#define RCC_OSC32_OUTC15_Pin` GPIO_PIN_15
- `#define RCC_OSC32_OUTC15_GPIO_Port` GPIOC
- `#define RCC_OSC_IN_Pin` GPIO_PIN_0
- `#define RCC_OSC_IN_GPIO_Port` GPIOF
- `#define RCC_OSC_OUT_Pin` GPIO_PIN_1
- `#define RCC_OSC_OUT_GPIO_Port` GPIOF
- `#define LPUART1_TX_Pin` GPIO_PIN_2
- `#define LPUART1_TX_GPIO_Port` GPIOA
- `#define LPUART1_RX_Pin` GPIO_PIN_3
- `#define LPUART1_RX_GPIO_Port` GPIOA
- `#define T_SWDIO_Pin` GPIO_PIN_13
- `#define T_SWDIO_GPIO_Port` GPIOA
- `#define T_SWCLK_Pin` GPIO_PIN_14
- `#define T_SWCLK_GPIO_Port` GPIOA
- `#define T_SWO_Pin` GPIO_PIN_3
- `#define T_SWO_GPIO_Port` GPIOB

Functions

- void [Error_Handler](#) (void)
This function is executed in case of error occurrence.

6.1.1 Detailed Description

: Header for [main.c](#) file. This file contains the common defines of the application.

Attention

Copyright (c) 2024 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

6.1.2 Function Documentation

6.1.2.1 Error_Handler()

```
void Error_Handler (
    void )
```

This function is executed in case of error occurrence.

Return values

None	
------	--

6.2 main.h

[Go to the documentation of this file.](#)

```
00001 /* USER CODE BEGIN Header */
00019 /* USER CODE END Header */
00020
00021 /* Define to prevent recursive inclusion -----*/
00022 #ifndef __MAIN_H
00023 #define __MAIN_H
00024
00025 #ifdef __cplusplus
00026 extern "C" {
00027 #endif
00028
00029 /* Includes -----*/
00030 #include "stm32g4xx_hal.h"
00031
00032 /* Private includes -----*/
00033 /* USER CODE BEGIN Includes */
00034
00035 /* USER CODE END Includes */
00036
00037 /* Exported types -----*/
00038 /* USER CODE BEGIN ET */
00039
00040 /* USER CODE END ET */
00041
00042 /* Exported constants -----*/
00043 /* USER CODE BEGIN EC */
00044
00045 /* USER CODE END EC */
00046
00047 /* Exported macro -----*/
00048 /* USER CODE BEGIN EM */
00049
00050 /* USER CODE END EM */
00051
```

```

00052 /* Exported functions prototypes -----*/
00053 void Error_Handler(void);
00054
00055 /* USER CODE BEGIN EFP */
00056
00057 /* USER CODE END EFP */
00058
00059 /* Private defines -----*/
00060 #define RCC_OSC32_OUT_Pin GPIO_PIN_14
00061 #define RCC_OSC32_OUT_GPIO_Port GPIOC
00062 #define RCC_OSC32_OUTC15_Pin GPIO_PIN_15
00063 #define RCC_OSC32_OUTC15_GPIO_Port GPIOC
00064 #define RCC_OSC_IN_Pin GPIO_PIN_0
00065 #define RCC_OSC_IN_GPIO_Port GPIOF
00066 #define RCC_OSC_OUT_Pin GPIO_PIN_1
00067 #define RCC_OSC_OUT_GPIO_Port GPIOF
00068 #define LPUART1_TX_Pin GPIO_PIN_2
00069 #define LPUART1_TX_GPIO_Port GPIOA
00070 #define LPUART1_RX_Pin GPIO_PIN_3
00071 #define LPUART1_RX_GPIO_Port GPIOA
00072 #define T_SWDIO_Pin GPIO_PIN_13
00073 #define T_SWDIO_GPIO_Port GPIOA
00074 #define T_SWCLK_Pin GPIO_PIN_14
00075 #define T_SWCLK_GPIO_Port GPIOA
00076 #define T_SWO_Pin GPIO_PIN_3
00077 #define T_SWO_GPIO_Port GPIOB
00078
00079 /* USER CODE BEGIN Private defines */
00080
00081 /* USER CODE END Private defines */
00082
00083 #ifdef __cplusplus
00084 }
00085 #endif
00086
00087 #endif /* __MAIN_H */

```

6.3 Core/Inc/stm32g4xx_hal_conf.h File Reference

HAL configuration file.

```

#include "stm32g4xx_hal_rcc.h"
#include "stm32g4xx_hal_gpio.h"
#include "stm32g4xx_hal_dma.h"
#include "stm32g4xx_hal_cortex.h"
#include "stm32g4xx_hal_exti.h"
#include "stm32g4xx_hal_flash.h"
#include "stm32g4xx_hal_i2c.h"
#include "stm32g4xx_hal_pwr.h"
#include "stm32g4xx_hal_tim.h"
#include "stm32g4xx_hal_uart.h"

```

Macros

- **#define HAL_MODULE_ENABLED**
This is the list of modules to be used in the HAL driver.
- **#define HAL_I2C_MODULE_ENABLED**
- **#define HAL_TIM_MODULE_ENABLED**
- **#define HAL_UART_MODULE_ENABLED**
- **#define HAL_GPIO_MODULE_ENABLED**
- **#define HAL_EXTI_MODULE_ENABLED**
- **#define HAL_DMA_MODULE_ENABLED**
- **#define HAL_RCC_MODULE_ENABLED**
- **#define HAL_FLASH_MODULE_ENABLED**

- `#define HAL_PWR_MODULE_ENABLED`
- `#define HAL_CORTEX_MODULE_ENABLED`
- `#define USE_HAL_ADC_REGISTER_CALLBACKS 0U`

This is the list of modules where register callback can be used.

- `#define USE_HAL_COMP_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_CORDIC_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_Cryp_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_DAC_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_EXTI_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_FDCAN_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_FMAC_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_HRTIM_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_I2C_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_I2S_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_IRDA_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_LPTIM_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_NAND_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_NOR_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_OPAMP_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_PCD_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_QSPI_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_RNG_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_RTC_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_SAI_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_SMARTCARD_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_SMBUS_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_SPI_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_SRAM_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_TIM_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_UART_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_USART_REGISTER_CALLBACKS 0U`
- `#define USE_HAL_WWDG_REGISTER_CALLBACKS 0U`
- `#define HSE_VALUE (24000000UL)`

Adjust the value of External High Speed oscillator (HSE) used in your application. This value is used by the RCC HAL module to compute the system frequency (when HSE is used as system clock source, directly or through the PLL).

- `#define HSE_STARTUP_TIMEOUT (100UL)`
- `#define HSI_VALUE (16000000UL)`

Internal High Speed oscillator (HSI) value. This value is used by the RCC HAL module to compute the system frequency (when HSI is used as system clock source, directly or through the PLL).

- `#define HSI48_VALUE (48000000UL)`

Internal High Speed oscillator (HSI48) value for USB FS and RNG. This internal oscillator is mainly dedicated to provide a high precision clock to the USB peripheral by means of a special Clock Recovery System (CRS) circuitry. When the CRS is not used, the HSI48 RC oscillator runs on it default frequency which is subject to manufacturing process variations.

- `#define LSI_VALUE (32000UL)`

Internal Low Speed oscillator (LSI) value.

- `#define LSE_VALUE (32768UL)`

External Low Speed oscillator (LSE) value. This value is used by the UART, RTC HAL module to compute the system frequency.

- `#define LSE_STARTUP_TIMEOUT (5000UL)`
- `#define EXTERNAL_CLOCK_VALUE (12288000UL)`

External clock source for I2S and SAI peripherals This value is used by the I2S and SAI HAL modules to compute the I2S and SAI clock source frequency, this source is inserted directly through I2S_CKIN pad.

- `#define VDD_VALUE (3300UL)`

This is the HAL system configuration section.

- #define `TICK_INT_PRIORITY` (0UL)
- #define `USE_RTOS` 0U
- #define `PREFETCH_ENABLE` 0U
- #define `INSTRUCTION_CACHE_ENABLE` 1U
- #define `DATA_CACHE_ENABLE` 1U
- #define `USE_SPI_CRC` 0U

Uncomment the line below to expanse the "assert_param" macro in the HAL drivers code.

- #define `assert_param`(expr)

Include module's header file.

6.3.1 Detailed Description

HAL configuration file.

Author

MCD Application Team

Attention

Copyright (c) 2019 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

6.3.2 Macro Definition Documentation

6.3.2.1 `assert_param`

```
#define assert_param(  
    expr)
```

Value:

```
((void)0U)
```

Include module's header file.

6.3.2.2 `EXTERNAL_CLOCK_VALUE`

```
#define EXTERNAL_CLOCK_VALUE (12288000UL)
```

External clock source for I2S and SAI peripherals This value is used by the I2S and SAI HAL modules to compute the I2S and SAI clock source frequency, this source is inserted directly through I2S_CKIN pad.

Value of the External oscillator in Hz

6.3.2.3 HSE_STARTUP_TIMEOUT

```
#define HSE_STARTUP_TIMEOUT (100UL)
```

Time out for HSE start up, in ms

6.3.2.4 HSE_VALUE

```
#define HSE_VALUE (24000000UL)
```

Adjust the value of External High Speed oscillator (HSE) used in your application. This value is used by the RCC HAL module to compute the system frequency (when HSE is used as system clock source, directly or through the PLL).

Value of the External oscillator in Hz

6.3.2.5 HSI48_VALUE

```
#define HSI48_VALUE (48000000UL)
```

Internal High Speed oscillator (HSI48) value for USB FS and RNG. This internal oscillator is mainly dedicated to provide a high precision clock to the USB peripheral by means of a special Clock Recovery System (CRS) circuitry. When the CRS is not used, the HSI48 RC oscillator runs on it default frequency which is subject to manufacturing process variations.

Value of the Internal High Speed oscillator for USB FS/RNG in Hz. The real value may vary depending on manufacturing process variations.

6.3.2.6 HSI_VALUE

```
#define HSI_VALUE (16000000UL)
```

Internal High Speed oscillator (HSI) value. This value is used by the RCC HAL module to compute the system frequency (when HSI is used as system clock source, directly or through the PLL).

Value of the Internal oscillator in Hz

6.3.2.7 LSE_STARTUP_TIMEOUT

```
#define LSE_STARTUP_TIMEOUT (5000UL)
```

Time out for LSE start up, in ms

6.3.2.8 LSE_VALUE

```
#define LSE_VALUE (32768UL)
```

External Low Speed oscillator (LSE) value. This value is used by the UART, RTC HAL module to compute the system frequency.

Value of the External Low Speed oscillator in Hz

6.3.2.9 LSI_VALUE

```
#define LSI_VALUE (32000UL)
```

Internal Low Speed oscillator (LSI) value.

< Value of the Internal Low Speed oscillator in Hz The real value may vary depending on the variations in voltage and temperature. LSI Typical Value in Hz

6.3.2.10 TICK_INT_PRIORITY

```
#define TICK_INT_PRIORITY (0UL)
```

tick interrupt priority (lowest by default)

6.3.2.11 VDD_VALUE

```
#define VDD_VALUE (3300UL)
```

This is the HAL system configuration section.

Value of VDD in mv

6.4 stm32g4xx_hal_conf.h

[Go to the documentation of this file.](#)

```
00001 /* USER CODE BEGIN Header */
00019 /* USER CODE END Header */
00020
00021 /* Define to prevent recursive inclusion -----*/
00022 #ifndef STM32G4xx_HAL_CONF_H
00023 #define STM32G4xx_HAL_CONF_H
00024
00025 #ifdef __cplusplus
00026 extern "C" {
00027 #endif
00028
00029 /* Exported types -----*/
00030 /* Exported constants -----*/
00031
00032 /* ##### Module Selection ##### */
00036
00037 #define HAL_MODULE_ENABLED
00038
00039 /*#define HAL_ADC_MODULE_ENABLED */
00040 /*#define HAL_COMP_MODULE_ENABLED */
00041 /*#define HAL_CORDIC_MODULE_ENABLED */
00042 /*#define HAL_CRC_MODULE_ENABLED */
00043 /*#define HAL_CRYP_MODULE_ENABLED */
00044 /*#define HAL_DAC_MODULE_ENABLED */
00045 /*#define HAL_FDCAN_MODULE_ENABLED */
00046 /*#define HAL_FMAC_MODULE_ENABLED */
00047 /*#define HAL_HRTIM_MODULE_ENABLED */
00048 /*#define HAL_IRDA_MODULE_ENABLED */
00049 /*#define HAL_IWDG_MODULE_ENABLED */
00050 #define HAL_I2C_MODULE_ENABLED
00051 /*#define HAL_I2S_MODULE_ENABLED */
00052 /*#define HAL_LPTIM_MODULE_ENABLED */
00053 /*#define HAL_NAND_MODULE_ENABLED */
00054 /*#define HAL_NOR_MODULE_ENABLED */
00055 /*#define HAL_OPAMP_MODULE_ENABLED */
00056 /*#define HAL_PCD_MODULE_ENABLED */
00057 /*#define HAL_QSPI_MODULE_ENABLED */
00058 /*#define HAL_RNG_MODULE_ENABLED */
```

```

00059 /**define HAL_RTC_MODULE_ENABLED */
00060 /**define HAL_SAI_MODULE_ENABLED */
00061 /**define HAL_SMARTCARD_MODULE_ENABLED */
00062 /**define HAL_SMBUS_MODULE_ENABLED */
00063 /**define HAL_SPI_MODULE_ENABLED */
00064 /**define HAL_SRAM_MODULE_ENABLED */
00065 #define HAL_TIM_MODULE_ENABLED
00066 #define HAL_UART_MODULE_ENABLED
00067 /**define HAL_USART_MODULE_ENABLED */
00068 /**define HAL_WWDG_MODULE_ENABLED */
00069 #define HAL_GPIO_MODULE_ENABLED
00070 #define HAL_EXTI_MODULE_ENABLED
00071 #define HAL_DMA_MODULE_ENABLED
00072 #define HAL_RCC_MODULE_ENABLED
00073 #define HAL_FLASH_MODULE_ENABLED
00074 #define HAL_PWR_MODULE_ENABLED
00075 #define HAL_CORTEX_MODULE_ENABLED
00076
00077 /* ##### Register Callbacks selection ##### */
00081 #define USE_HAL_ADC_REGISTER_CALLBACKS 0U
00082 #define USE_HAL_COMP_REGISTER_CALLBACKS 0U
00083 #define USE_HAL_CORDIC_REGISTER_CALLBACKS 0U
00084 #define USE_HAL_Cryp_REGISTER_CALLBACKS 0U
00085 #define USE_HAL_DAC_REGISTER_CALLBACKS 0U
00086 #define USE_HAL_EXTI_REGISTER_CALLBACKS 0U
00087 #define USE_HAL_FDCAN_REGISTER_CALLBACKS 0U
00088 #define USE_HAL_FMAC_REGISTER_CALLBACKS 0U
00089 #define USE_HAL_HRTIM_REGISTER_CALLBACKS 0U
00090 #define USE_HAL_I2C_REGISTER_CALLBACKS 0U
00091 #define USE_HAL_I2S_REGISTER_CALLBACKS 0U
00092 #define USE_HAL_IRDA_REGISTER_CALLBACKS 0U
00093 #define USE_HAL_LPTIM_REGISTER_CALLBACKS 0U
00094 #define USE_HAL_NAND_REGISTER_CALLBACKS 0U
00095 #define USE_HAL_NOR_REGISTER_CALLBACKS 0U
00096 #define USE_HAL_OPAMP_REGISTER_CALLBACKS 0U
00097 #define USE_HAL_PCD_REGISTER_CALLBACKS 0U
00098 #define USE_HAL_QSPI_REGISTER_CALLBACKS 0U
00099 #define USE_HAL_RNG_REGISTER_CALLBACKS 0U
00100 #define USE_HAL_RTC_REGISTER_CALLBACKS 0U
00101 #define USE_HAL_SAI_REGISTER_CALLBACKS 0U
00102 #define USE_HAL_SMARTCARD_REGISTER_CALLBACKS 0U
00103 #define USE_HAL_SMBUS_REGISTER_CALLBACKS 0U
00104 #define USE_HAL_SPI_REGISTER_CALLBACKS 0U
00105 #define USE_HAL_SRAM_REGISTER_CALLBACKS 0U
00106 #define USE_HAL_TIM_REGISTER_CALLBACKS 0U
00107 #define USE_HAL_UART_REGISTER_CALLBACKS 0U
00108 #define USE_HAL_USART_REGISTER_CALLBACKS 0U
00109 #define USE_HAL_WWDG_REGISTER_CALLBACKS 0U
00110
00111 /* ##### Oscillator Values adaptation #####*/
00117 #if !defined (HSE_VALUE)
00118 #define HSE_VALUE (24000000UL)
00119 #endif /* HSE_VALUE */
00120
00121 #if !defined (HSE_STARTUP_TIMEOUT)
00122 #define HSE_STARTUP_TIMEOUT (100UL)
00123 #endif /* HSE_STARTUP_TIMEOUT */
00124
00130 #if !defined (HSI_VALUE)
00131 #define HSI_VALUE (16000000UL)
00132 #endif /* HSI_VALUE */
00133
00141 #if !defined (HSI48_VALUE)
00142 #define HSI48_VALUE (48000000UL)
00144 #endif /* HSI48_VALUE */
00145
00149 #if !defined (LSI_VALUE)
00152 #define LSI_VALUE (32000UL)
00153 #endif /* LSI_VALUE */
00158 #if !defined (LSE_VALUE)
00159 #define LSE_VALUE (32768UL)
00160 #endif /* LSE_VALUE */
00161
00162 #if !defined (LSE_STARTUP_TIMEOUT)
00163 #define LSE_STARTUP_TIMEOUT (5000UL)
00164 #endif /* LSE_STARTUP_TIMEOUT */
00165
00171 #if !defined (EXTERNAL_CLOCK_VALUE)
00172 #define EXTERNAL_CLOCK_VALUE (12288000UL)
00173 #endif /* EXTERNAL_CLOCK_VALUE */
00174
00175 /* Tip: To avoid modifying this file each time you need to use different HSE,
00176 == you can define the HSE value in your toolchain compiler preprocessor. */
00177
00178 /* ##### System Configuration ##### */
00182
00183 #define VDD_VALUE (3300UL)

```

```

00184 #define TICK_INT_PRIORITY          (0UL)
00185 #define USE_RTOS                     0U
00186 #define PREFETCH_ENABLE              0U
00187 #define INSTRUCTION_CACHE_ENABLE     1U
00188 #define DATA_CACHE_ENABLE           1U
00189
00190 /* ##### Assert Selection ##### */
00191 /* #define USE_FULL_ASSERT    1U */
00192
00193 /* ##### SPI peripheral configuration ##### */
00194
00195 /* CRC FEATURE: Use to activate CRC feature inside HAL SPI Driver
00196  * Activated: CRC code is present inside driver
00197  * Deactivated: CRC code cleaned from driver
00198  */
00199
00200 #define USE_SPI_CRC                  0U
00201
00202 /* Includes -----*/
00203
00204 #ifndef HAL_RCC_MODULE_ENABLED
00205 #include "stm32g4xx_hal_rcc.h"
00206 #endif /* HAL_RCC_MODULE_ENABLED */
00207
00208 #ifndef HAL_GPIO_MODULE_ENABLED
00209 #include "stm32g4xx_hal_gpio.h"
00210 #endif /* HAL_GPIO_MODULE_ENABLED */
00211
00212 #ifndef HAL_DMA_MODULE_ENABLED
00213 #include "stm32g4xx_hal_dma.h"
00214 #endif /* HAL_DMA_MODULE_ENABLED */
00215
00216 #ifndef HAL_CORTEX_MODULE_ENABLED
00217 #include "stm32g4xx_hal_cortex.h"
00218 #endif /* HAL_CORTEX_MODULE_ENABLED */
00219
00220 #ifndef HAL_ADC_MODULE_ENABLED
00221 #include "stm32g4xx_hal_adc.h"
00222 #endif /* HAL_ADC_MODULE_ENABLED */
00223
00224 #ifndef HAL_COMP_MODULE_ENABLED
00225 #include "stm32g4xx_hal_comp.h"
00226 #endif /* HAL_COMP_MODULE_ENABLED */
00227
00228 #ifndef HAL_CORDIC_MODULE_ENABLED
00229 #include "stm32g4xx_hal_cordic.h"
00230 #endif /* HAL_CORDIC_MODULE_ENABLED */
00231
00232 #ifndef HAL_CRC_MODULE_ENABLED
00233 #include "stm32g4xx_hal_crc.h"
00234 #endif /* HAL_CRC_MODULE_ENABLED */
00235
00236 #ifndef HAL_CRYP_MODULE_ENABLED
00237 #include "stm32g4xx_hal_cryp.h"
00238 #endif /* HAL_CRYP_MODULE_ENABLED */
00239
00240 #ifndef HAL_DAC_MODULE_ENABLED
00241 #include "stm32g4xx_hal_dac.h"
00242 #endif /* HAL_DAC_MODULE_ENABLED */
00243
00244 #ifndef HAL_EXTI_MODULE_ENABLED
00245 #include "stm32g4xx_hal_exti.h"
00246 #endif /* HAL_EXTI_MODULE_ENABLED */
00247
00248 #ifndef HAL_FDCAN_MODULE_ENABLED
00249 #include "stm32g4xx_hal_fdcan.h"
00250 #endif /* HAL_FDCAN_MODULE_ENABLED */
00251
00252 #ifndef HAL_FLASH_MODULE_ENABLED
00253 #include "stm32g4xx_hal_flash.h"
00254 #endif /* HAL_FLASH_MODULE_ENABLED */
00255
00256 #ifndef HAL_FMAC_MODULE_ENABLED
00257 #include "stm32g4xx_hal_fmac.h"
00258 #endif /* HAL_FMAC_MODULE_ENABLED */
00259
00260 #ifndef HAL_HRTIM_MODULE_ENABLED
00261 #include "stm32g4xx_hal_hrtim.h"
00262 #endif /* HAL_HRTIM_MODULE_ENABLED */
00263
00264 #ifndef HAL_IRDA_MODULE_ENABLED
00265 #include "stm32g4xx_hal_irda.h"
00266 #endif /* HAL_IRDA_MODULE_ENABLED */
00267
00268 #ifndef HAL_IWDG_MODULE_ENABLED
00269 #include "stm32g4xx_hal_iwdg.h"
00270 #endif /* HAL_IWDG_MODULE_ENABLED */

```

```
00278
00279 #ifndef HAL_I2C_MODULE_ENABLED
00280 #include "stm32g4xx_hal_i2c.h"
00281 #endif /* HAL_I2C_MODULE_ENABLED */
00282
00283 #ifndef HAL_I2S_MODULE_ENABLED
00284 #include "stm32g4xx_hal_i2s.h"
00285 #endif /* HAL_I2S_MODULE_ENABLED */
00286
00287 #ifndef HAL_LPTIM_MODULE_ENABLED
00288 #include "stm32g4xx_hal_lptim.h"
00289 #endif /* HAL_LPTIM_MODULE_ENABLED */
00290
00291 #ifndef HAL_NAND_MODULE_ENABLED
00292 #include "stm32g4xx_hal_nand.h"
00293 #endif /* HAL_NAND_MODULE_ENABLED */
00294
00295 #ifndef HAL_NOR_MODULE_ENABLED
00296 #include "stm32g4xx_hal_nor.h"
00297 #endif /* HAL_NOR_MODULE_ENABLED */
00298
00299 #ifndef HAL_OPAMP_MODULE_ENABLED
00300 #include "stm32g4xx_hal_opamp.h"
00301 #endif /* HAL_OPAMP_MODULE_ENABLED */
00302
00303 #ifndef HAL_PCD_MODULE_ENABLED
00304 #include "stm32g4xx_hal_pcd.h"
00305 #endif /* HAL_PCD_MODULE_ENABLED */
00306
00307 #ifndef HAL_PWR_MODULE_ENABLED
00308 #include "stm32g4xx_hal_pwr.h"
00309 #endif /* HAL_PWR_MODULE_ENABLED */
00310
00311 #ifndef HAL_QSPI_MODULE_ENABLED
00312 #include "stm32g4xx_hal_qspi.h"
00313 #endif /* HAL_QSPI_MODULE_ENABLED */
00314
00315 #ifndef HAL_RNG_MODULE_ENABLED
00316 #include "stm32g4xx_hal_rng.h"
00317 #endif /* HAL_RNG_MODULE_ENABLED */
00318
00319 #ifndef HAL_RTC_MODULE_ENABLED
00320 #include "stm32g4xx_hal_rtc.h"
00321 #endif /* HAL_RTC_MODULE_ENABLED */
00322
00323 #ifndef HAL_SAI_MODULE_ENABLED
00324 #include "stm32g4xx_hal_sai.h"
00325 #endif /* HAL_SAI_MODULE_ENABLED */
00326
00327 #ifndef HAL_SMARTCARD_MODULE_ENABLED
00328 #include "stm32g4xx_hal_smartcard.h"
00329 #endif /* HAL_SMARTCARD_MODULE_ENABLED */
00330
00331 #ifndef HAL_SMBUS_MODULE_ENABLED
00332 #include "stm32g4xx_hal_smbus.h"
00333 #endif /* HAL_SMBUS_MODULE_ENABLED */
00334
00335 #ifndef HAL_SPI_MODULE_ENABLED
00336 #include "stm32g4xx_hal_spi.h"
00337 #endif /* HAL_SPI_MODULE_ENABLED */
00338
00339 #ifndef HAL_SRAM_MODULE_ENABLED
00340 #include "stm32g4xx_hal_sram.h"
00341 #endif /* HAL_SRAM_MODULE_ENABLED */
00342
00343 #ifndef HAL_TIM_MODULE_ENABLED
00344 #include "stm32g4xx_hal_tim.h"
00345 #endif /* HAL_TIM_MODULE_ENABLED */
00346
00347 #ifndef HAL_UART_MODULE_ENABLED
00348 #include "stm32g4xx_hal_uart.h"
00349 #endif /* HAL_UART_MODULE_ENABLED */
00350
00351 #ifndef HAL_USART_MODULE_ENABLED
00352 #include "stm32g4xx_hal_usart.h"
00353 #endif /* HAL_USART_MODULE_ENABLED */
00354
00355 #ifndef HAL_WWDG_MODULE_ENABLED
00356 #include "stm32g4xx_hal_wwdg.h"
00357 #endif /* HAL_WWDG_MODULE_ENABLED */
00358
00359 /* Exported macro -----*/
00360 #ifndef USE_FULL_ASSERT
00361 #define assert_param(expr) ((expr) ? (void)0U : assert_failed((uint8_t *)__FILE__, __LINE__))
00362 #endif /* Exported functions ----- */
00363 void assert_failed(uint8_t *file, uint32_t line);
00364 #else
```

```
00373 #define assert_param(expr) ((void)0U)
00374 #endif /* USE_FULL_ASSERT */
00375
00376 #ifdef __cplusplus
00377 }
00378 #endif
00379
00380 #endif /* STM32G4xx_HAL_CONF_H */
```

6.5 Core/Inc/stm32g4xx_it.h File Reference

This file contains the headers of the interrupt handlers.

Functions

- void **NMI_Handler** (void)
This function handles Non maskable interrupt.
- void **HardFault_Handler** (void)
This function handles Hard fault interrupt.
- void **MemManage_Handler** (void)
This function handles Memory management fault.
- void **BusFault_Handler** (void)
This function handles Prefetch fault, memory access fault.
- void **UsageFault_Handler** (void)
This function handles Undefined instruction or illegal state.
- void **SVC_Handler** (void)
This function handles System service call via SWI instruction.
- void **DebugMon_Handler** (void)
This function handles Debug monitor.
- void **PendSV_Handler** (void)
This function handles Pendable request for system service.
- void **SysTick_Handler** (void)
This function handles System tick timer.
- void **TIM2_IRQHandler** (void)
This function handles TIM2 global interrupt.

6.5.1 Detailed Description

This file contains the headers of the interrupt handlers.

Attention

Copyright (c) 2024 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

6.6 stm32g4xx_it.h

[Go to the documentation of this file.](#)

```

00001 /* USER CODE BEGIN Header */
00018 /* USER CODE END Header */
00019
00020 /* Define to prevent recursive inclusion -----*/
00021 #ifndef __STM32G4xx_IT_H
00022 #define __STM32G4xx_IT_H
00023
00024 #ifdef __cplusplus
00025 extern "C" {
00026 #endif
00027
00028 /* Private includes -----*/
00029 /* USER CODE BEGIN Includes */
00030
00031 /* USER CODE END Includes */
00032
00033 /* Exported types -----*/
00034 /* USER CODE BEGIN ET */
00035
00036 /* USER CODE END ET */
00037
00038 /* Exported constants -----*/
00039 /* USER CODE BEGIN EC */
00040
00041 /* USER CODE END EC */
00042
00043 /* Exported macro -----*/
00044 /* USER CODE BEGIN EM */
00045
00046 /* USER CODE END EM */
00047
00048 /* Exported functions prototypes -----*/
00049 void NMI_Handler(void);
00050 void HardFault_Handler(void);
00051 void MemManage_Handler(void);
00052 void BusFault_Handler(void);
00053 void UsageFault_Handler(void);
00054 void SVC_Handler(void);
00055 void DebugMon_Handler(void);
00056 void PendSV_Handler(void);
00057 void SysTick_Handler(void);
00058 void TIM2_IRQHandler(void);
00059 /* USER CODE BEGIN EFP */
00060
00061 /* USER CODE END EFP */
00062
00063 #ifdef __cplusplus
00064 }
00065 #endif
00066
00067 #endif /* __STM32G4xx_IT_H */

```

6.7 Core/Src/main.c File Reference

: Main program body

```

#include "main.h"
#include "tools.h"
#include "functions.h"
#include "compass.h"

```

Functions

- void [SystemClock_Config](#) (void)
System Clock Configuration.
- static void [MX_GPIO_Init](#) (void)

- GPIO Initialization Function.*
 - static void [MX_I2C1_Init](#) (void)
 - I2C1 Initialization Function.*
 - static void [MX_LPUART1_UART_Init](#) (void)
 - LPUART1 Initialization Function.*
 - static void [MX_TIM2_Init](#) (void)
 - TIM2 Initialization Function.*
 - int [main](#) (void)
 - The application entry point.*
 - void [Error_Handler](#) (void)
- This function is executed in case of error occurrence.*

Variables

- I2C_HandleTypeDef [hi2c1](#)
- UART_HandleTypeDef [hlpuart1](#)
- TIM_HandleTypeDef [htim2](#)
- [CompassCalib_t](#) [acc_calib](#)

6.7.1 Detailed Description

: Main program body

Attention

Copyright (c) 2024 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

6.7.2 Function Documentation

6.7.2.1 Error_Handler()

```
void Error_Handler (  
    void )
```

This function is executed in case of error occurrence.

Return values

<i>None</i>	
-------------	--

6.7.2.2 main()

```
int main (  
    void )
```

The application entry point.

Return values

<i>int</i>	
------------	--

6.7.2.3 MX_GPIO_Init()

```
void MX_GPIO_Init (
    void ) [static]
```

GPIO Initialization Function.

Parameters

<i>None</i>	
-------------	--

Return values

<i>None</i>	
-------------	--

6.7.2.4 MX_I2C1_Init()

```
void MX_I2C1_Init (
    void ) [static]
```

I2C1 Initialization Function.

Parameters

<i>None</i>	
-------------	--

Return values

<i>None</i>	
-------------	--

Configure Analogue filter

Configure Digital filter

6.7.2.5 MX_LPUART1_UART_Init()

```
void MX_LPUART1_UART_Init (
    void ) [static]
```

LPUART1 Initialization Function.

Parameters

--	--

<i>None</i>	
-------------	--

Return values

<i>None</i>	
-------------	--

6.7.2.6 MX_TIM2_Init()

```
void MX_TIM2_Init (
    void ) [static]
```

TIM2 Initialization Function.

Parameters

<i>None</i>	
-------------	--

Return values

<i>None</i>	
-------------	--

6.7.2.7 SystemClock_Config()

```
void SystemClock_Config (
    void )
```

System Clock Configuration.

Return values

<i>None</i>	
-------------	--

Configure the main internal regulator output voltage

Initializes the RCC Oscillators according to the specified parameters in the RCC_OscInitTypeDef structure.

Initializes the CPU, AHB and APB buses clocks

6.8 Core/Src/stm32g4xx_hal_msp.c File Reference

This file provides code for the MSP Initialization and de-Initialization codes.

```
#include "main.h"
```

Functions

- void [HAL_MspInit](#) (void)
- void [HAL_I2C_MspInit](#) (I2C_HandleTypeDef *hi2c)
I2C MSP Initialization This function configures the hardware resources used in this example.
- void [HAL_I2C_MspDeInit](#) (I2C_HandleTypeDef *hi2c)
I2C MSP De-Initialization This function freeze the hardware resources used in this example.
- void [HAL_UART_MspInit](#) (UART_HandleTypeDef *huart)
UART MSP Initialization This function configures the hardware resources used in this example.
- void [HAL_UART_MspDeInit](#) (UART_HandleTypeDef *huart)
UART MSP De-Initialization This function freeze the hardware resources used in this example.
- void [HAL_TIM_Base_MspInit](#) (TIM_HandleTypeDef *htim_base)
TIM_Base MSP Initialization This function configures the hardware resources used in this example.
- void [HAL_TIM_Base_MspDeInit](#) (TIM_HandleTypeDef *htim_base)
TIM_Base MSP De-Initialization This function freeze the hardware resources used in this example.

6.8.1 Detailed Description

This file provides code for the MSP Initialization and de-Initialization codes.

Attention

Copyright (c) 2024 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

6.8.2 Function Documentation

6.8.2.1 HAL_I2C_MspDeInit()

```
void HAL_I2C_MspDeInit (
    I2C_HandleTypeDef * hi2c)
```

I2C MSP De-Initialization This function freeze the hardware resources used in this example.

Parameters

<i>hi2c</i>	I2C handle pointer
-------------	--------------------

Return values

<i>None</i>	
-------------	--

I2C1 GPIO Configuration PA15 -----> I2C1_SCL PB7 -----> I2C1_SDA

6.8.2.2 HAL_I2C_MspInit()

```
void HAL_I2C_MspInit (  
    I2C_HandleTypeDef * hi2c)
```

I2C MSP Initialization This function configures the hardware resources used in this example.

Parameters

<i>hi2c</i>	I2C handle pointer
-------------	--------------------

Return values

<i>None</i>	
-------------	--

Initializes the peripherals clocks

I2C1 GPIO Configuration PA15 -----> I2C1_SCL PB7 -----> I2C1_SDA

6.8.2.3 HAL_MspInit()

```
void HAL_MspInit (
    void )
```

Initializes the Global MSP. Disable the internal Pull-Up in Dead Battery pins of UCPD peripheral

6.8.2.4 HAL_TIM_Base_MspDeInit()

```
void HAL_TIM_Base_MspDeInit (
    TIM_HandleTypeDef * htim_base)
```

TIM_Base MSP De-Initialization This function freeze the hardware resources used in this example.

Parameters

<i>htim_base</i>	TIM_Base handle pointer
------------------	-------------------------

Return values

<i>None</i>	
-------------	--

6.8.2.5 HAL_TIM_Base_MspInit()

```
void HAL_TIM_Base_MspInit (
    TIM_HandleTypeDef * htim_base)
```

TIM_Base MSP Initialization This function configures the hardware resources used in this example.

Parameters

<i>htim_base</i>	TIM_Base handle pointer
------------------	-------------------------

Return values

<i>None</i>	
-------------	--

6.8.2.6 HAL_UART_MspDeInit()

```
void HAL_UART_MspDeInit (
    UART_HandleTypeDef * huart)
```

UART MSP De-Initialization This function freeze the hardware resources used in this example.

Parameters

<i>huart</i>	UART handle pointer
--------------	---------------------

Return values

<i>None</i>	
-------------	--

LPUART1 GPIO Configuration PA2 -----> LPUART1_TX PA3 -----> LPUART1_RX

6.8.2.7 HAL_UART_MspInit()

```
void HAL_UART_MspInit (
    UART_HandleTypeDef * huart)
```

UART MSP Initialization This function configures the hardware resources used in this example.

Parameters

<i>huart</i>	UART handle pointer
--------------	---------------------

Return values

<i>None</i>	
-------------	--

Initializes the peripherals clocks

LPUART1 GPIO Configuration PA2 -----> LPUART1_TX PA3 -----> LPUART1_RX

6.9 Core/Src/stm32g4xx_it.c File Reference

Interrupt Service Routines.

```
#include "main.h"
#include "stm32g4xx_it.h"
```

Functions

- void **NMI_Handler** (void)
This function handles Non maskable interrupt.
- void **HardFault_Handler** (void)
This function handles Hard fault interrupt.
- void **MemManage_Handler** (void)
This function handles Memory management fault.
- void **BusFault_Handler** (void)
This function handles Prefetch fault, memory access fault.
- void **UsageFault_Handler** (void)
This function handles Undefined instruction or illegal state.
- void **SVC_Handler** (void)
This function handles System service call via SWI instruction.
- void **DebugMon_Handler** (void)
This function handles Debug monitor.
- void **PendSV_Handler** (void)
This function handles Pendable request for system service.
- void **SysTick_Handler** (void)
This function handles System tick timer.
- void **TIM2_IRQHandler** (void)
This function handles TIM2 global interrupt.

Variables

- TIM_HandleTypeDef **htim2**

6.9.1 Detailed Description

Interrupt Service Routines.

Attention

Copyright (c) 2024 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

6.10 Core/Src/syscalls.c File Reference

STM32CubeIDE Minimal System calls file.

```
#include <sys/stat.h>
#include <stdlib.h>
#include <errno.h>
#include <stdio.h>
#include <signal.h>
#include <time.h>
#include <sys/time.h>
#include <sys/times.h>
```

Functions

- int **__io_putchar** (int ch) __attribute__((weak))
- int **__io_getchar** (void)
- void **initialise_monitor_handles** ()
- int **_getpid** (void)
- int **_kill** (int pid, int sig)
- void **_exit** (int status)
- **__attribute__** ((weak))
- int **_close** (int file)
- int **_fstat** (int file, struct stat *st)
- int **_isatty** (int file)
- int **_lseek** (int file, int ptr, int dir)
- int **_open** (char *path, int flags,...)
- int **_wait** (int *status)
- int **_unlink** (char *name)
- int **_times** (struct tms *buf)
- int **_stat** (char *file, struct stat *st)
- int **_link** (char *old, char *new)
- int **_fork** (void)
- int **_execve** (char *name, char **argv, char **env)

Variables

- char ** **environ** = __env

6.10.1 Detailed Description

STM32CubeIDE Minimal System calls file.

Author

Auto-generated by STM32CubeIDE

```
For more information about which c-functions
need which of these lowlevel functions
please consult the Newlib libc-manual
```

Attention

Copyright (c) 2020-2024 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

6.11 Core/Src/sysmem.c File Reference

STM32CubeIDE System Memory calls file.

```
#include <errno.h>
#include <stdint.h>
```

Functions

- void * [_sbrk](#) (ptrdiff_t incr)
[_sbrk\(\)](#) allocates memory to the newlib heap and is used by malloc and others from the C library

Variables

- static uint8_t * [__sbrk_heap_end](#) = NULL

6.11.1 Detailed Description

STM32CubeIDE System Memory calls file.

Author

Generated by STM32CubeIDE

For more information about which C functions
need which of these lowlevel functions
please consult the newlib libc manual

Attention

Copyright (c) 2024 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

6.11.2 Function Documentation

6.11.2.1 [_sbrk\(\)](#)

```
void * \_sbrk (
    ptrdiff_t incr)
```

[_sbrk\(\)](#) allocates memory to the newlib heap and is used by malloc and others from the C library

```
* #####
* # .data # .bss #          newlib heap          #          MSP stack          #
* #          #          #          #          # Reserved by _Min_Stack_Size #
* #####
* ^-- RAM start          ^-- _end          _estack, RAM end --^
*
```

This implementation starts allocating at the '_end' linker symbol The '_Min_Stack_Size' linker symbol reserves a memory for the MSP stack The implementation considers '_estack' linker symbol to be RAM end NOTE: If the MSP stack, at any point during execution, grows larger than the reserved size, please increase the '_Min_Stack_Size'.

Parameters

<i>incr</i>	Memory size
-------------	-------------

Returns

Pointer to allocated memory

6.11.3 Variable Documentation**6.11.3.1 __sbrk_heap_end**

```
uint8_t* __sbrk_heap_end = NULL [static]
```

Pointer to the current high watermark of the heap usage

6.12 Core/Src/system_stm32g4xx.c File Reference

CMSIS Cortex-M4 Device Peripheral Access Layer System Source File.

```
#include "stm32g4xx.h"
```

Macros

- #define **HSE_VALUE** 24000000U
- #define **HSI_VALUE** 16000000U

Functions

- void **SystemInit** (void)
Setup the microcontroller system.
- void **SystemCoreClockUpdate** (void)
Update SystemCoreClock variable according to Clock Register Values. The SystemCoreClock variable contains the core clock (HCLK), it can be used by the user application to setup the SysTick timer or configure other parameters.

Variables

- uint32_t **SystemCoreClock** = **HSI_VALUE**
- const uint8_t **AHBPrescTable** [16] = {0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U, 1U, 2U, 3U, 4U, 6U, 7U, 8U, 9U}
- const uint8_t **APBPrescTable** [8] = {0U, 0U, 0U, 0U, 1U, 2U, 3U, 4U}

6.12.1 Detailed Description

CMSIS Cortex-M4 Device Peripheral Access Layer System Source File.

Author

MCD Application Team

This file provides two functions and one global variable to be called from user application:

- [SystemInit\(\)](#): This function is called at startup just after reset and before branch to main program. This call is made inside the "startup_stm32g4xx.s" file.
- SystemCoreClock variable: Contains the core clock (HCLK), it can be used by the user application to setup the SysTick timer or configure other parameters.
- [SystemCoreClockUpdate\(\)](#): Updates the variable SystemCoreClock and must be called whenever the core clock is changed during program execution.

After each device reset the HSI (16 MHz) is used as system clock source. Then [SystemInit\(\)](#) function is called, in "startup_stm32g4xx.s" file, to configure the system clock before to branch to main program.

6.12.2 This file configures the system clock as follows:

6.12.2.1 System Clock source | HSI

6.12.2.2 SYSCLK(Hz) | 16000000

6.12.2.3 HCLK(Hz) | 16000000

6.12.2.4 AHB Prescaler | 1

6.12.2.5 APB1 Prescaler | 1

6.12.2.6 APB2 Prescaler | 1

6.12.2.7 PLL_M | 1

6.12.2.8 PLL_N | 16

6.12.2.9 PLL_P | 7

6.12.2.10 PLL_Q | 2

6.12.2.11 PLL_R | 2

6.12.2.12 Require 48MHz for RNG | Disabled

=====

Attention

Copyright (c) 2019 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

6.13 Utils/Inc/compass.h File Reference

En-têtes des fonctions pour la boussole électronique.

```
#include <stddef.h>
#include "main.h"
```

Classes

- struct [CompassCalib_t](#)

Macros

- `#define PI 3.14`

Functions

- void **CompassMeasureMsg** (const double acc[3], char *msg, size_t msg_len, [CompassCalib_t](#) *calib)
Calcule et formate les angles d'inclinaison à partir des données de l'accéléromètre.
- void **CompassCalibSinglePoint** (const double acc[3], [CompassCalib_t](#) *calib)
Calibre un accéléromètre avec une méthode simple single-point.
- void **TiltCompensatedCompass** (const double acc[3], const double mag[3], double *roll, double *pitch, double *heading)
Calcule l'orientation complète compensée en inclinaison (acc + mag) (identique à ta version, conservée).
- void [TiltCompensatedComplementary](#) (const double acc[3], const double gyro_rates[3], const double mag[3], double *roll, double *pitch, double *heading)
(NOUVEAU) Filtre complémentaire réel en discrete-time.
- void [TiltCompensatedCompassAngles](#) (const double acc[3], const double mag[3], double *roll, double *pitch, double *heading)
Ancienne fonction TiltCompensatedCompassAngles. Conservée comme fallback : applique une lissage simple sur les angles calculés uniquement à partir de l'accéléromètre (sans gyro).

Variables

- double **Te**
- double **tau**
- double **K**

6.13.1 Detailed Description

En-têtes des fonctions pour la boussole électronique.

6.13.2 Function Documentation

6.13.2.1 TiltCompensatedCompassAngles()

```
void TiltCompensatedCompassAngles (
    const double acc[3],
    const double mag[3],
    double * roll,
    double * pitch,
    double * heading)
```

Ancienne fonction TiltCompensatedCompassAngles. Conservée comme fallback : applique une lissage simple sur les angles calculés uniquement à partir de l'accéléromètre (sans gyro).

Pour un vrai filtre complémentaire (acc+gyro) utilise la fonction TiltCompensatedComplementary(...) ci-dessus.

6.13.2.2 TiltCompensatedComplementary()

```
void TiltCompensatedComplementary (
    const double acc[3],
    const double gyro_rates[3],
    const double mag[3],
    double * roll,
    double * pitch,
    double * heading)
```

(NOUVEAU) Filtre complémentaire réel en discrete-time.

```
theta[n] = K * (theta[n-1] + Te * omega[n]) + (1-K) * theta_acc[n]
```

Parameters

<i>acc</i>	accéléromètre calibré [ax,ay,az] en g
<i>gyro_rates</i>	gyroscope [gx,gy,gz] en deg/s ← Vérifie l'unité de tes lectures !
<i>mag</i>	magnétomètre calibré [mx,my,mz] en μ T (peut être NULL si tu ne veux pas heading)
<i>roll</i>	sortie roulis (deg)
<i>pitch</i>	sortie tangage (deg)
<i>heading</i>	sortie azimut nord magnétique (deg) – si mag == NULL, heading = NAN

6.14 compass.h

[Go to the documentation of this file.](#)

```
00001
00005
00006 #ifndef COMPASS_H
00007 #define COMPASS_H
00008
00009 #include <stddef.h>    // pour size_t
00010 #include "main.h"     // pour types STM32
00011
00012 // Structure pour stocker les offsets et gains
00013 typedef struct {
00014     double offset[3];
00015     double gain[3];
00016 } CompassCalib_t;
00017
00018 #define PI 3.14
00019
00020 /* Déclarations des paramètres (définis dans compass.c) */
00021 extern double Te;      // Période d'échantillonnage
00022 extern double tau;     // Constante de temps
00023 extern double K;       // Coefficient complémentaire Te)
00024
00025 /* Fonctions */
00026 void CompassMeasureMsg(const double acc[3], char *msg, size_t msg_len, CompassCalib_t *calib);
00027 void CompassCalibSinglePoint(const double acc[3], CompassCalib_t *calib);
00028
00029 /* Calculs de la boussole / inclinaison */
00030 void TiltCompensatedCompass(const double acc[3], const double mag[3], double *roll, double *pitch,
00031                             double *heading);
00032
00033 /*
00034  * Filtre complémentaire réel : fournit roll/pitch en utilisant acc + gyro
00035  * - gyro_rates : tableau [gx, gy, gz] en deg/s (ou rad/s si tes lectures le sont -- adapte si besoin)
00036  * - acc : accéléromètre calibré en g
00037  * - mag : magnétomètre calibré en  $\mu$ T (peut être NULL si on ne calcule pas heading ici)
00038  * Résultat : roll, pitch, heading (deg). Heading calculé via mag + compensation tilt.
00039  */
00039 void TiltCompensatedComplementary(const double acc[3], const double gyro_rates[3], const double
00040                                   mag[3],
00041                                   double *roll, double *pitch, double *heading);
00042
00043 void TiltCompensatedCompassAngles(const double acc[3], const double mag[3], double *roll, double
00044                                   *pitch, double *heading);
00051
00052 #endif /* COMPASS_H */
00053
```

6.15 Utils/Inc/functions.h File Reference

En-têtes des fonctions de gestion du MPU-9250.

```
#include "stm32g4xx_hal.h"
#include <stdint.h>
```

Macros

- #define **MPU9250_ADDRESS** 0x68 << 1
- #define **TEMP_OFFSET** 17
- #define **CALIB_MAG_SAMPLES** 300
- #define **CALIB_MAG_DELAY_MS** 50

Functions

- void **checkMPU9250_ID** (void)
Lecture du registre WHO_AM_I du MPU-9250 pour vérifier sa présence.
- void **InitSensors** (void)
Initialise le MPU-9250 et le magnétomètre AK8963.
- double **TempMeasureMsg** (char *buffer, size_t size)
Mesure la température et écrit le résultat dans un buffer.
- void **AccMeasureMsg** (double *acc_data, char *buffer, size_t size)
Mesure l'accélération selon les 3 axes du MPU-9250.
- void **GyroMeasureMsg** (double *gyro_data, char *buffer, size_t size)
Mesure la vitesse angulaire sur les 3 axes du MPU-9250.
- void **MagMeasureMsg** (double *mag_data, char *buffer, size_t size)
Mesure le champ magnétique 3D via le magnétomètre AK8963.
- double **ReadAccelSensitivity** (void)
Lit la sensibilité réelle de l'accéléromètre MPU-9250.
- double **ReadGyroSensitivity** (void)
Lit la sensibilité réelle du gyroscope MPU-9250.
- double **ReadMagSensitivity** (uint8_t bit16_mode)
Lit la sensibilité nominale du magnétomètre AK8963 selon le mode de sortie.
- void **TestAccelSensZ** (void)
Teste l'accéléromètre sur toutes les plages de sensibilité et affiche Z.
- void **GyrNoise** (double *gyro_rms, uint16_t nb_samples)
Calcule le bruit RMS du gyroscope sur les 3 axes.
- void **TestGyroNoise** (uint16_t nb_samples)
Teste le bruit RMS du gyroscope pour différentes bandes passantes.
- void **SetGyroBandwidth** (uint8_t bw_code)
- void **Average** (float data[][3], int N, float avg[3])
Calcule la moyenne de N mesures gyroscopiques.
- void **Variance** (float data[][3], int N, float avg[3], float var[3])
Calcule la variance des N mesures gyroscopiques.
- void **GyrCalib** (uint16_t N)
Calibre le gyroscope en mode "single point" (no turn).
- void **ReadGyro** (float *gx, float *gy, float *gz)
Lit les vitesses angulaires brutes (%s) sur les 3 axes.
- void **ReadOffsetGyro** (void)
Lit les offsets matériels du gyroscope dans le MPU-9250 et les affiche.
- void **ReadMag** (float *mx, float *my, float *mz)
Lit le champ magnétique 3D directement depuis le magnétomètre AK8963 sans afficher de message UART.
- void **MagAcquireSamples** (float *Bx, float *By, float *Bz, uint16_t N, uint16_t delay_ms)
Lit N mesures du magnétomètre et les stocke dans les tableaux Bx, By, Bz.
- void **MagComputeOffsets** (float *Bx, float *By, float *Bz, uint16_t N, float *offsetX, float *offsetY, float *offsetZ)
Calcule les offsets moyens (hard iron) à partir des mesures.
- void **MagPrintOffsets** (float offsetX, float offsetY, float offsetZ)

Affiche les offsets via UART.

- void [MagCalibLite](#) (uint16_t N, uint16_t delay_ms)

Fonction complète pour la calibration « light » du magnétomètre : acquisition des mesures, calcul des offsets et affichage.

- void [MagCalib](#) (I2C_HandleTypeDef *hi2c, double mag_bias[3], double mag_scale[3])

Calibre le magnétomètre pour compenser les hard iron et soft iron biases.

- void [MagMeasureMsgCalib](#) (double *mag_data, char *buffer, size_t size, const double mag_bias[3], const double mag_scale[3])

Mesure le champ magnétique corrigé par calibration (hard + soft iron).

6.15.1 Detailed Description

En-têtes des fonctions de gestion du MPU-9250.

6.15.2 Function Documentation

6.15.2.1 AccMeasureMsg()

```
void AccMeasureMsg (
    double * acc_data,
    char * buffer,
    size_t size)
```

Mesure l'accélération selon les 3 axes du MPU-9250.

Lit les registres ACCEL_XOUT_H/L (0x3B/0x3C), ACCEL_YOUT_H/L (0x3D/0x3E), ACCEL_ZOUT_H/L (0x3F/0x40) via I2C et convertit les valeurs brutes en g.

Les valeurs brutes sont codées en complément à 2 (int16_t). Pour l'étendue par défaut $\pm 2g$, la conversion est : $ax = ACCEL_X_RAW / 16384.0$ $ay = ACCEL_Y_RAW / 16384.0$ $az = ACCEL_Z_RAW / 16384.0$

La fonction affiche également les valeurs mesurées sur le terminal UART.

Parameters

<i>acc_data</i>	Pointeur vers un tableau de 3 doubles pour stocker [ax, ay, az] en g.
-----------------	---

6.15.2.2 Average()

```
void Average (
    float data[][3],
    int N,
    float avg[3])
```

Calcule la moyenne de N mesures gyroscopiques.

Parameters

<i>data</i>	Tableau contenant les mesures (X, Y, Z)
<i>N</i>	Nombre d'échantillons
<i>avg</i>	Tableau de sortie [3] pour stocker les moyennes

6.15.2.3 checkMPU9250_ID()

```
void checkMPU9250_ID (
    void )
```

Lecture du registre WHO_AM_I du MPU-9250 pour vérifier sa présence.

Lit le registre 0x75 du capteur (adresse 0x68) via le bus I2C. Affiche la valeur lue sur le terminal et allume la LED rouge en cas d'erreur.

Parameters

<i>none</i>	
-------------	--

Return values

<i>none</i>	
-------------	--

6.15.2.4 GyrCalib()

```
void GyrCalib (
    uint16_t N)
```

Calibre le gyroscope en mode "single point" (no turn).

Cette fonction mesure N échantillons lorsque le capteur est immobile, calcule les offsets pour chaque axe et les écrit dans les registres matériels de compensation du MPU-9250.

Parameters

<i>N</i>	Nombre d'échantillons utilisés pour la moyenne
----------	--

6.15.2.5 GyrNoise()

```
void GyrNoise (
    double * gyro_rms,
    uint16_t nb_samples)
```

Calcule le bruit RMS du gyroscope sur les 3 axes.

Lit nb_samples valeurs du gyroscope, convertit en %s et calcule le RMS.

Parameters

out	<i>gyro_rms</i>	Tableau de 3 doubles [gx, gy, gz] pour stocker les valeurs RMS.
in	<i>nb_samples</i>	Nombre d'échantillons à lire.

6.15.2.6 GyroMeasureMsg()

```
void GyroMeasureMsg (
    double * gyro_data,
    char * buffer,
    size_t size)
```

Mesure la vitesse angulaire sur les 3 axes du MPU-9250.

Lit les registres GYRO_XOUT_H/L, GYRO_YOUT_H/L, GYRO_ZOUT_H/L et convertit les valeurs brutes en %s.

Parameters

<i>gyro_data</i>	Pointeur vers un tableau de 3 doubles [gx, gy, gz] en %s
------------------	--

6.15.2.7 InitSensors()

```
void InitSensors (
    void )
```

Initialise le MPU-9250 et le magnétomètre AK8963.
 Cette fonction effectue les opérations suivantes :

1. Reset du MPU-9250
2. Sélection de l'horloge PLL interne
3. Lecture et vérification du registre WHO_AM_I du MPU-9250
4. Configuration du DLPF et du Sample Rate Divider pour ODR 3,91 Hz
5. Activation du mode BYPASS I2C pour accéder au magnétomètre
6. Lecture et vérification du registre WHO_AM_I du magnétomètre AK8963
7. Configuration du magnétomètre : 16 bits, Continuous Measurement Mode 2 (100 Hz)

Note

L'ODR du MPU-9250 est fixée à sa plus faible valeur (3,91 Hz) en utilisant `SMPLRT_DIV = 255` et `DLPF_CFG = 6`.

6.15.2.8 MagAcquireSamples()

```
void MagAcquireSamples (
    float * Bx,
    float * By,
    float * Bz,
    uint16_t N,
    uint16_t delay_ms)
```

Lit N mesures du magnétomètre et les stocke dans les tableaux Bx, By, Bz.

Parameters

<i>Bx</i>	Tableau de float pour stocker les mesures X
<i>By</i>	Tableau de float pour stocker les mesures Y
<i>Bz</i>	Tableau de float pour stocker les mesures Z
<i>N</i>	Nombre de mesures à effectuer
<i>delay_ms</i>	Délai entre chaque mesure en millisecondes

6.15.2.9 MagCalib()

```
void MagCalib (
    I2C_HandleTypeDef * hi2c,
    double mag_bias[3],
    double mag_scale[3])
```

Calibre le magnétomètre pour compenser les hard iron et soft iron biases.

Parameters

<i>hi2c</i>	Pointeur vers la structure I2C (ex: &hi2c1)
<i>mag_bias</i>	Tableau de 3 doubles pour stocker les offsets (hard iron) calculés
<i>mag_scale</i>	Tableau de 3 doubles pour stocker les coefficients correcteurs (soft iron) calculés

Return values

<i>None</i>	
-------------	--

Note

Utilise [ReadMag\(\)](#) pour ne pas afficher de messages UART inutiles.

6.15.2.10 MagCalibLite()

```
void MagCalibLite (
    uint16_t N,
    uint16_t delay_ms)
```

Fonction complète pour la calibration « light » du magnétomètre : acquisition des mesures, calcul des offsets et affichage.

Parameters

<i>N</i>	Nombre de mesures à acquérir
<i>delay_ms</i>	Délai entre chaque mesure en millisecondes

6.15.2.11 MagComputeOffsets()

```
void MagComputeOffsets (
    float * Bx,
    float * By,
    float * Bz,
    uint16_t N,
    float * offsetX,
    float * offsetY,
    float * offsetZ)
```

Calcule les offsets moyens (hard iron) à partir des mesures.

Parameters

<i>Bx</i>	Tableau des mesures X
<i>By</i>	Tableau des mesures Y
<i>Bz</i>	Tableau des mesures Z
<i>N</i>	Nombre de mesures
<i>offsetX</i>	Pointeur vers float pour stocker l'offset X moyen
<i>offsetY</i>	Pointeur vers float pour stocker l'offset Y moyen
<i>offsetZ</i>	Pointeur vers float pour stocker l'offset Z moyen

6.15.2.12 MagMeasureMsg()

```
void MagMeasureMsg (
    double * mag_data,
    char * buffer,
    size_t size)
```

Mesure le champ magnétique 3D via le magnétomètre AK8963.

Lit les registres HXL à HZH (0x03 à 0x08), applique les coefficients d'ajustement (ASAX, ASAY, ASAZ), puis convertit les valeurs brutes en μT .

Parameters

<i>mag_data</i>	Pointeur vers un tableau de 3 doubles [Bx, By, Bz] (μ T)
<i>buffer</i>	Pointeur vers la chaîne de caractères à remplir
<i>size</i>	Taille du buffer

6.15.2.13 MagMeasureMsgCalib()

```
void MagMeasureMsgCalib (
    double * mag_data,
    char * buffer,
    size_t size,
    const double mag_bias[3],
    const double mag_scale[3])
```

Mesure le champ magnétique corrigé par calibration (hard + soft iron).

Cette fonction applique les offsets (hard iron) et les coefficients de correction (soft iron) calculés lors de [MagCalib\(\)](#).

Puis elle affiche dans la commande les valeurs brutes et celles calibrées.

Parameters

<i>mag_data</i>	Pointeur vers un tableau de 3 doubles [Bx, By, Bz] (μ T)
<i>buffer</i>	Pointeur vers la chaîne de caractères à remplir (UART)
<i>size</i>	Taille du buffer
<i>mag_bias</i>	Tableau [3] des offsets calculés
<i>mag_scale</i>	Tableau [3] des coefficients correcteurs

6.15.2.14 MagPrintOffsets()

```
void MagPrintOffsets (
    float offsetX,
    float offsetY,
    float offsetZ)
```

Affiche les offsets via UART.

Parameters

<i>offsetX</i>	Offset moyen sur l'axe X
<i>offsetY</i>	Offset moyen sur l'axe Y
<i>offsetZ</i>	Offset moyen sur l'axe Z

6.15.2.15 ReadAccelSensitivity()

```
double ReadAccelSensitivity (
    void )
```

Lit la sensibilité réelle de l'accéléromètre MPU-9250.

Lit le registre ACCEL_CONFIG et retourne le facteur de conversion en LSB/g pour la plage configurée.

Returns

double Sensibilité (LSB/g)

6.15.2.16 ReadGyro()

```
void ReadGyro (
    float * gx,
    float * gy,
    float * gz)
```

Lit les vitesses angulaires brutes (%s) sur les 3 axes.

Parameters

<i>gx</i>	Pointeur vers la valeur X
<i>gy</i>	Pointeur vers la valeur Y
<i>gz</i>	Pointeur vers la valeur Z

6.15.2.17 ReadGyroSensitivity()

```
double ReadGyroSensitivity (
    void )
```

Lit la sensibilité réelle du gyroscope MPU-9250.

Returns

double Sensibilité (LSB/(%s)) selon la plage configurée

6.15.2.18 ReadMag()

```
void ReadMag (
    float * mx,
    float * my,
    float * mz)
```

Lit le champ magnétique 3D directement depuis le magnétomètre AK8963 sans afficher de message UART.

Parameters

<i>mx</i>	Pointeur vers float pour stocker l'axe X (μ T)
<i>my</i>	Pointeur vers float pour stocker l'axe Y (μ T)
<i>mz</i>	Pointeur vers float pour stocker l'axe Z (μ T)

6.15.2.19 ReadMagSensitivity()

```
double ReadMagSensitivity (
    uint8_t bit16_mode)
```

Lit la sensibilité nominale du magnétomètre AK8963 selon le mode de sortie.

Parameters

<i>bit16_mode</i>	Si 1, mode 16 bits (0.15 μ T/LSB). Si 0, mode 14 bits (0.6 μ T/LSB).
-------------------	--

Returns

double Sensibilité du magnétomètre en μ T/LSB

6.15.2.20 ReadOffsetGyro()

```
void ReadOffsetGyro (
    void )
```

Lit les offsets matériels du gyroscope dans le MPU-9250 et les affiche.

Note

Permet de vérifier si les offsets écrits sont bien proches de 0 après calibration.

Return values

<i>None</i>	
-------------	--

6.15.2.21 TempMeasureMsg()

```
double TempMeasureMsg (
    char * buffer,
    size_t size)
```

Mesure la température et écrit le résultat dans un buffer.

Parameters

<i>buffer</i>	pointeur vers char[] pour recevoir le message
<i>size</i>	taille du buffer

Returns

double température en °C

6.15.2.22 TestAccelSensZ()

```
void TestAccelSensZ (
    void )
```

Teste l'accéléromètre sur toutes les plages de sensibilité et affiche Z.

Affiche : FS_SEL, plage (g), sensibilité (LSB/g), Z_raw et Z_g

6.15.2.23 TestGyroNoise()

```
void TestGyroNoise (
    uint16_t nb_samples)
```

Teste le bruit RMS du gyroscope pour différentes bandes passantes.

Parameters

<i>nb_samples</i>	Nombre de mesures pour calculer le RMS
-------------------	--

6.15.2.24 Variance()

```
void Variance (
    float data[][3],
    int N,
    float avg[3],
    float var[3])
```

Calcule la variance des N mesures gyroscopiques.

Parameters

<i>data</i>	Tableau [N][3]
<i>N</i>	Nombre d'échantillons
<i>avg</i>	Moyennes calculées pour chaque axe
<i>var</i>	Tableau de sortie [3] pour stocker les variances

6.16 functions.h

[Go to the documentation of this file.](#)

```

00001
00005
00006 #ifndef INC_FUNCTIONS_H_
00007 #define INC_FUNCTIONS_H_
00008
00009 #include "stm32g4xx_hal.h"
00010 #include <stdint.h>
00011
00012 void checkMPU9250_ID(void);
00013 void InitSensors(void);
00014
00015 double TempMeasureMsg(char *buffer, size_t size);
00016 void AccMeasureMsg(double *acc_data, char *buffer, size_t size);
00017 void GyroMeasureMsg(double *gyro_data, char *buffer, size_t size);
00018 void MagMeasureMsg(double *mag_data, char *buffer, size_t size);
00019
00020 double ReadAccelSensitivity(void);
00021 double ReadGyroSensitivity(void);
00022 double ReadMagSensitivity(uint8_t bit16_mode);
00023
00024 void TestAccelSensZ(void);
00025 void GyrNoise(double *gyro_rms, uint16_t nb_samples);
00026 void TestGyroNoise(uint16_t nb_samples);
00027 void SetGyroBandwidth(uint8_t bw_code);
00028
00029 void Average(float data[][3], int N, float avg[3]);
00030 void Variance(float data[][3], int N, float avg[3], float var[3]);
00031 void GyrCalib(uint16_t N);
00032 void ReadGyro(float *gx, float *gy, float *gz);
00033 void ReadOffsetGyro(void);
00034
00035 void ReadMag(float *mx, float *my, float *mz);
00036 void MagAcquireSamples(float *Bx, float *By, float *Bz, uint16_t N, uint16_t delay_ms);
00037 void MagComputeOffsets(float *Bx, float *By, float *Bz, uint16_t N, float *offsetX, float *offsetY,
00038 float *offsetZ);
00038 void MagPrintOffsets(float offsetX, float offsetY, float offsetZ);
00039 void MagCalibLite(uint16_t N, uint16_t delay_ms);
00040 void MagCalib(I2C_HandleTypeDef *hi2c, double mag_bias[3], double mag_scale[3]);
00041 void MagMeasureMsgCalib(double *mag_data, char *buffer, size_t size, const double mag_bias[3], const
00042 double mag_scale[3]);
00042
00043 #define MPU9250_ADDRESS 0x68 << 1
00044 #define TEMP_OFFSET 17
00045 #define CALIB_MAG_SAMPLES 300
00046 #define CALIB_MAG_DELAY_MS 50
00047
00048 #endif /* INC_FUNCTIONS_H_ */

```

6.17 Utils/Inc/tools.h File Reference

En-têtes des fonctions de tout début de projet.

```

#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include "stm32g4xx_hal.h"
#include "stm32g4xx_hal_gpio.h"

```

Functions

- void **setup** ()

Initialisation UART + message d'accueil. Affiche un message de bienvenue sur le terminal série et prépare l'environnement de test.

- void **loop** ()
Boucle principale : clignotement LED + affichage compteur UART.
- void **test_LED** (void)
Test du fonctionnement des LEDs verte et rouge.

6.17.1 Detailed Description

En-têtes des fonctions de tout début de projet.

6.17.2 Function Documentation

6.17.2.1 test_LED()

```
void test_LED (
    void )
```

Test du fonctionnement des LEDs verte et rouge.

Allume la LED pendant 1 seconde puis l'éteint. Permet de vérifier la configuration GPIO et la fonction HAL_Delay.

6.18 tools.h

[Go to the documentation of this file.](#)

```
00001
00005
00006 #ifndef INC_TOOLS_H_
00007 #define INC_TOOLS_H_
00008 #include <stdio.h>
00009 #include <stdint.h>
00010 #include <string.h>
00011 #include "stm32g4xx_hal.h"
00012 #include "stm32g4xx_hal_gpio.h"
00013
00014 void setup();
00015 void loop();
00016 void test_LED(void);
00017
00018
00019
00020 #endif /* INC_TOOLS_H_ */
```

6.19 Utils/Src/compass.c File Reference

Création d'une boussole électronique (mono et multi capteurs).

```
#include "compass.h"
#include <math.h>
#include <stdio.h>
```

Macros

- #define **PI** 3.14159265358979323846

Functions

- static void **CompassInitParameters** (void)
Initialise K si nécessaire.
- void **CompassCalibSinglePoint** (const double acc[3], [CompassCalib_t](#) *calib)
Calibre un accéléromètre avec une méthode simple single-point.
- void **CompassMeasureMsg** (const double acc[3], char *msg, size_t msg_len, [CompassCalib_t](#) *calib)
Calcule et formate les angles d'inclinaison à partir des données de l'accéléromètre.
- void **TiltCompensatedCompass** (const double acc[3], const double mag[3], double *roll, double *pitch, double *heading)

Calcule l'orientation complète compensée en inclinaison (acc + mag) (identique à ta version, conservée).

- void [TiltCompensatedCompassAngles](#) (const double acc[3], const double mag[3], double *roll, double *pitch, double *heading)

Ancienne fonction TiltCompensatedCompassAngles. Conservée comme fallback : applique une lissage simple sur les angles calculés uniquement à partir de l'accéléromètre (sans gyro).

- void [TiltCompensatedComplementary](#) (const double acc[3], const double gyro_rates[3], const double mag[3], double *roll, double *pitch, double *heading)

(NOUVEAU) Filtre complémentaire réel en discrete-time.

Variables

- double **Te** = 0.01
- double **tau** = 0.1
- double **K** = 0.0

6.19.1 Detailed Description

Création d'une boussole électronique (mono et multi capteurs).

Author

Kevin

Date

Novembre 2025

6.19.2 Function Documentation

6.19.2.1 TiltCompensatedCompassAngles()

```
void TiltCompensatedCompassAngles (
    const double acc[3],
    const double mag[3],
    double * roll,
    double * pitch,
    double * heading)
```

Ancienne fonction TiltCompensatedCompassAngles. Conservée comme fallback : applique une lissage simple sur les angles calculés uniquement à partir de l'accéléromètre (sans gyro).

Pour un vrai filtre complémentaire (acc+gyro) utilise la fonction TiltCompensatedComplementary(...) ci-dessus.

6.19.2.2 TiltCompensatedComplementary()

```
void TiltCompensatedComplementary (
    const double acc[3],
    const double gyro_rates[3],
    const double mag[3],
    double * roll,
    double * pitch,
    double * heading)
```

(NOUVEAU) Filtre complémentaire réel en discrete-time.

$$\text{theta}[n] = K * (\text{theta}[n-1] + \text{Te} * \text{omega}[n]) + (1-K) * \text{theta_acc}[n]$$

Parameters

<i>acc</i>	accéléromètre calibré [ax,ay,az] en g
------------	---------------------------------------

<i>gyro_rates</i>	gyroscope [gx,gy,gz] en deg/s <— Vérifie l'unité de tes lectures !
<i>mag</i>	magnétomètre calibré [mx,my,mz] en μ T (peut être NULL si tu ne veux pas heading)
<i>roll</i>	sortie roulis (deg)
<i>pitch</i>	sortie tangage (deg)
<i>heading</i>	sortie azimuth nord magnétique (deg) – si mag == NULL, heading = NAN

6.20 Utils/Src/functions.c File Reference

Fonctions de gestion du capteur MPU-9250 (IMU 9 axes).

```
#include "functions.h"
#include "tools.h"
#include "stm32g4xx_hal.h"
#include <stdio.h>
#include <string.h>
#include <stdint.h>
#include <math.h>
#include <float.h>
```

Functions

- void [Error_Handler](#) (void)
This function is executed in case of error occurrence.
- void [MagMeasureMsg](#) (double *mag_data, char *buffer, size_t size)
Measure le champ magnétique 3D via le magnétomètre AK8963.
- void [checkMPU9250_ID](#) (void)
Lecture du registre WHO_AM_I du MPU-9250 pour vérifier sa présence.
- void [InitSensors](#) (void)
Initialise le MPU-9250 et le magnétomètre AK8963.
- double [TempMeasureMsg](#) (char *buffer, size_t size)
Mesure la température et écrit le résultat dans un buffer.
- double [ReadAccelSensitivity](#) (void)
Lit la sensibilité réelle de l'accéléromètre MPU-9250.
- void [AccMeasureMsg](#) (double *acc_data, char *buffer, size_t size)
Mesure l'accélération selon les 3 axes du MPU-9250.
- void [TestAccelSensZ](#) (void)
Teste l'accéléromètre sur toutes les plages de sensibilité et affiche Z.
- double [ReadGyroSensitivity](#) (void)
Lit la sensibilité réelle du gyroscope MPU-9250.
- void [GyroMeasureMsg](#) (double *gyro_data, char *buffer, size_t size)
Mesure la vitesse angulaire sur les 3 axes du MPU-9250.
- void [GyrNoise](#) (double *gyro_rms, uint16_t nb_samples)
Calcule le bruit RMS du gyroscope sur les 3 axes.
- void [TestGyroNoise](#) (uint16_t nb_samples)
Teste le bruit RMS du gyroscope pour différentes bandes passantes.
- void [SetGyroBandwidth](#) (uint8_t bw_code)
- double [ReadMagSensitivity](#) (uint8_t bit16_mode)
Lit la sensibilité nominale du magnétomètre AK8963 selon le mode de sortie.
- void [Average](#) (float data[][3], int N, float avg[3])
Calcule la moyenne de N mesures gyroscopiques.
- void [Variance](#) (float data[][3], int N, float avg[3], float var[3])
Calcule la variance des N mesures gyroscopiques.

- void [ReadGyro](#) (float *gx, float *gy, float *gz)
Lit les vitesses angulaires brutes (%s) sur les 3 axes.
- void [GyrCalib](#) (uint16_t N)
Calibre le gyroscope en mode "single point" (no turn).
- void [ReadOffsetGyro](#) (void)
Lit les offsets matériels du gyroscope dans le MPU-9250 et les affiche.
- void [ReadMag](#) (float *mx, float *my, float *mz)
Lit le champ magnétique 3D directement depuis le magnétomètre AK8963 sans afficher de message UART.
- void [MagAcquireSamples](#) (float *Bx, float *By, float *Bz, uint16_t N, uint16_t delay_ms)
Lit N mesures du magnétomètre et les stocke dans les tableaux Bx, By, Bz.
- void [MagComputeOffsets](#) (float *Bx, float *By, float *Bz, uint16_t N, float *offsetX, float *offsetY, float *offsetZ)
Calcule les offsets moyens (hard iron) à partir des mesures.
- void [MagPrintOffsets](#) (float offsetX, float offsetY, float offsetZ)
Affiche les offsets via UART.
- void [MagCalibLite](#) (uint16_t N, uint16_t delay_ms)
Fonction complète pour la calibration « light » du magnétomètre : acquisition des mesures, calcul des offsets et affichage.
- void [MagCalib](#) (I2C_HandleTypeDef *hi2c, double mag_bias[3], double mag_scale[3])
Calibre le magnétomètre pour compenser les hard iron et soft iron biases.
- void [MagMeasureMsgCalib](#) (double *mag_data, char *buffer, size_t size, const double mag_bias[3], const double mag_scale[3])
Mesure le champ magnétique corrigé par calibration (hard + soft iron).

Variables

- I2C_HandleTypeDef **hi2c1**
- UART_HandleTypeDef **hlpuart1**
- uint8_t **message** [100]

6.20.1 Detailed Description

Fonctions de gestion du capteur MPU-9250 (IMU 9 axes).

Author

Kevin

Date

Novembre 2025

6.20.2 Function Documentation

6.20.2.1 AccMeasureMsg()

```
void AccMeasureMsg (
    double * acc_data,
    char * buffer,
    size_t size)
```

Mesure l'accélération selon les 3 axes du MPU-9250.

Lit les registres ACCEL_XOUT_H/L (0x3B/0x3C), ACCEL_YOUT_H/L (0x3D/0x3E), ACCEL_ZOUT_H/L (0x3F/0x40) via I2C et convertit les valeurs brutes en g.

Les valeurs brutes sont codées en complément à 2 (int16_t). Pour l'étendue par défaut ±2g, la conversion est : ax = ACCEL_X_RAW / 16384.0 ay = ACCEL_Y_RAW / 16384.0 az = ACCEL_Z_RAW / 16384.0

La fonction affiche également les valeurs mesurées sur le terminal UART.

Parameters

<i>acc_data</i>	Pointeur vers un tableau de 3 doubles pour stocker [ax, ay, az] en g.
-----------------	---

6.20.2.2 Average()

```
void Average (
    float data[][3],
    int N,
    float avg[3])
```

Calcule la moyenne de N mesures gyroscopiques.

Parameters

<i>data</i>	Tableau contenant les mesures (X, Y, Z)
<i>N</i>	Nombre d'échantillons
<i>avg</i>	Tableau de sortie [3] pour stocker les moyennes

6.20.2.3 checkMPU9250_ID()

```
void checkMPU9250_ID (
    void )
```

Lecture du registre WHO_AM_I du MPU-9250 pour vérifier sa présence.

Lit le registre 0x75 du capteur (adresse 0x68) via le bus I2C. Affiche la valeur lue sur le terminal et allume la LED rouge en cas d'erreur.

Parameters

<i>none</i>	
-------------	--

Return values

<i>none</i>	
-------------	--

6.20.2.4 Error_Handler()

```
void Error_Handler (
    void ) [extern]
```

This function is executed in case of error occurrence.

Return values

<i>None</i>	
-------------	--

6.20.2.5 GyrCalib()

```
void GyrCalib (
    uint16_t N)
```

Calibre le gyroscope en mode "single point" (no turn).

Cette fonction mesure N échantillons lorsque le capteur est immobile, calcule les offsets pour chaque axe et les écrit dans les registres matériels de compensation du MPU-9250.

Parameters

<i>N</i>	Nombre d'échantillons utilisés pour la moyenne
----------	--

6.20.2.6 GyrNoise()

```
void GyrNoise (
    double * gyro_rms,
    uint16_t nb_samples)
```

Calcule le bruit RMS du gyroscope sur les 3 axes.

Lit *nb_samples* valeurs du gyroscope, convertit en %s et calcule le RMS.

Parameters

out	<i>gyro_rms</i>	Tableau de 3 doubles [gx, gy, gz] pour stocker les valeurs RMS.
in	<i>nb_samples</i>	Nombre d'échantillons à lire.

6.20.2.7 GyroMeasureMsg()

```
void GyroMeasureMsg (
    double * gyro_data,
    char * buffer,
    size_t size)
```

Mesure la vitesse angulaire sur les 3 axes du MPU-9250.

Lit les registres GYRO_XOUT_H/L, GYRO_YOUT_H/L, GYRO_ZOUT_H/L et convertit les valeurs brutes en %s.

Parameters

<i>gyro_data</i>	Pointeur vers un tableau de 3 doubles [gx, gy, gz] en %s
------------------	--

6.20.2.8 InitSensors()

```
void InitSensors (
    void )
```

Initialise le MPU-9250 et le magnétomètre AK8963.

Cette fonction effectue les opérations suivantes :

1. Reset du MPU-9250
2. Sélection de l'horloge PLL interne
3. Lecture et vérification du registre WHO_AM_I du MPU-9250
4. Configuration du DLPF et du Sample Rate Divider pour ODR 3,91 Hz
5. Activation du mode BYPASS I2C pour accéder au magnétomètre
6. Lecture et vérification du registre WHO_AM_I du magnétomètre AK8963
7. Configuration du magnétomètre : 16 bits, Continuous Measurement Mode 2 (100 Hz)

Note

L'ODR du MPU-9250 est fixée à sa plus faible valeur (3,91 Hz) en utilisant `SMPLRT_DIV = 255` et `DLPF_CFG = 6`.

6.20.2.9 MagAcquireSamples()

```
void MagAcquireSamples (
    float * Bx,
    float * By,
    float * Bz,
    uint16_t N,
    uint16_t delay_ms)
```

Lit N mesures du magnétomètre et les stocke dans les tableaux Bx, By, Bz.

Parameters

<i>Bx</i>	Tableau de float pour stocker les mesures X
<i>By</i>	Tableau de float pour stocker les mesures Y
<i>Bz</i>	Tableau de float pour stocker les mesures Z
<i>N</i>	Nombre de mesures à effectuer
<i>delay_ms</i>	Délai entre chaque mesure en millisecondes

6.20.2.10 MagCalib()

```
void MagCalib (
    I2C_HandleTypeDef * hi2c,
    double mag_bias[3],
    double mag_scale[3])
```

Calibre le magnétomètre pour compenser les hard iron et soft iron biases.

Parameters

<i>hi2c</i>	Pointeur vers la structure I2C (ex: &hi2c1)
<i>mag_bias</i>	Tableau de 3 doubles pour stocker les offsets (hard iron) calculés
<i>mag_scale</i>	Tableau de 3 doubles pour stocker les coefficients correcteurs (soft iron) calculés

Return values

<i>None</i>	
-------------	--

Note

Utilise [ReadMag\(\)](#) pour ne pas afficher de messages UART inutiles.

6.20.2.11 MagCalibLite()

```
void MagCalibLite (
    uint16_t N,
    uint16_t delay_ms)
```

Fonction complète pour la calibration « light » du magnétomètre : acquisition des mesures, calcul des offsets et affichage.

Parameters

<i>N</i>	Nombre de mesures à acquérir
<i>delay_ms</i>	Délai entre chaque mesure en millisecondes

6.20.2.12 MagComputeOffsets()

```
void MagComputeOffsets (
    float * Bx,
    float * By,
    float * Bz,
    uint16_t N,
    float * offsetX,
    float * offsetY,
    float * offsetZ)
```

Calcule les offsets moyens (hard iron) à partir des mesures.

Parameters

<i>Bx</i>	Tableau des mesures X
<i>By</i>	Tableau des mesures Y
<i>Bz</i>	Tableau des mesures Z
<i>N</i>	Nombre de mesures
<i>offsetX</i>	Pointeur vers float pour stocker l'offset X moyen
<i>offsetY</i>	Pointeur vers float pour stocker l'offset Y moyen
<i>offsetZ</i>	Pointeur vers float pour stocker l'offset Z moyen

6.20.2.13 MagMeasureMsg()

```
void MagMeasureMsg (
    double * mag_data,
    char * buffer,
    size_t size) [extern]
```

Mesure le champ magnétique 3D via le magnétomètre AK8963.

Lit les registres HXL à HZH (0x03 à 0x08), applique les coefficients d'ajustement (ASAX, ASAY, ASAZ), puis convertit les valeurs brutes en μT .

Parameters

<i>mag_data</i>	Pointeur vers un tableau de 3 doubles [Bx, By, Bz] (μT)
<i>buffer</i>	Pointeur vers la chaîne de caractères à remplir
<i>size</i>	Taille du buffer

6.20.2.14 MagMeasureMsgCalib()

```
void MagMeasureMsgCalib (
    double * mag_data,
    char * buffer,
    size_t size,
    const double mag_bias[3],
    const double mag_scale[3])
```

Mesure le champ magnétique corrigé par calibration (hard + soft iron).

Cette fonction applique les offsets (hard iron) et les coefficients de correction (soft iron) calculés lors de [MagCalib\(\)](#). Puis elle affiche dans la commande les valeurs brutes et celles calibrées.

Parameters

<i>mag_data</i>	Pointeur vers un tableau de 3 doubles [Bx, By, Bz] (μ T)
<i>buffer</i>	Pointeur vers la chaîne de caractères à remplir (UART)
<i>size</i>	Taille du buffer
<i>mag_bias</i>	Tableau [3] des offsets calculés
<i>mag_scale</i>	Tableau [3] des coefficients correcteurs

6.20.2.15 MagPrintOffsets()

```
void MagPrintOffsets (
    float offsetX,
    float offsetY,
    float offsetZ)
```

Affiche les offsets via UART.

Parameters

<i>offsetX</i>	Offset moyen sur l'axe X
<i>offsetY</i>	Offset moyen sur l'axe Y
<i>offsetZ</i>	Offset moyen sur l'axe Z

6.20.2.16 ReadAccelSensitivity()

```
double ReadAccelSensitivity (
    void )
```

Lit la sensibilité réelle de l'accéléromètre MPU-9250.

Lit le registre ACCEL_CONFIG et retourne le facteur de conversion en LSB/g pour la plage configurée.

Returns

double Sensibilité (LSB/g)

6.20.2.17 ReadGyro()

```
void ReadGyro (
    float * gx,
    float * gy,
    float * gz)
```

Lit les vitesses angulaires brutes (%/s) sur les 3 axes.

Parameters

<i>gx</i>	Pointeur vers la valeur X
<i>gy</i>	Pointeur vers la valeur Y
<i>gz</i>	Pointeur vers la valeur Z

6.20.2.18 ReadGyroSensitivity()

```
double ReadGyroSensitivity (
    void )
```

Lit la sensibilité réelle du gyroscope MPU-9250.

Returns

double Sensibilité (LSB/(%/s)) selon la plage configurée

6.20.2.19 ReadMag()

```
void ReadMag (
    float * mx,
    float * my,
    float * mz)
```

Lit le champ magnétique 3D directement depuis le magnétomètre AK8963 sans afficher de message UART.

Parameters

<i>mx</i>	Pointeur vers float pour stocker l'axe X (μ T)
<i>my</i>	Pointeur vers float pour stocker l'axe Y (μ T)
<i>mz</i>	Pointeur vers float pour stocker l'axe Z (μ T)

6.20.2.20 ReadMagSensitivity()

```
double ReadMagSensitivity (
    uint8_t bit16_mode)
```

Lit la sensibilité nominale du magnétomètre AK8963 selon le mode de sortie.

Parameters

<i>bit16_mode</i>	Si 1, mode 16 bits (0.15 μ T/LSB). Si 0, mode 14 bits (0.6 μ T/LSB).
-------------------	--

Returns

double Sensibilité du magnétomètre en μ T/LSB

6.20.2.21 ReadOffsetGyro()

```
void ReadOffsetGyro (
    void )
```

Lit les offsets matériels du gyroscope dans le MPU-9250 et les affiche.

Note

Permet de vérifier si les offsets écrits sont bien proches de 0 après calibration.

Return values

<i>None</i>	
-------------	--

6.20.2.22 TempMeasureMsg()

```
double TempMeasureMsg (
    char * buffer,
    size_t size)
```

Mesure la température et écrit le résultat dans un buffer.

Parameters

<i>buffer</i>	pointeur vers char[] pour recevoir le message
<i>size</i>	taille du buffer

Returns

double température en °C

6.20.2.23 TestAccelSensZ()

```
void TestAccelSensZ (
    void )
```

Teste l'accéléromètre sur toutes les plages de sensibilité et affiche Z.
Affiche : FS_SEL, plage (g), sensibilité (LSB/g), Z_raw et Z_g

6.20.2.24 TestGyroNoise()

```
void TestGyroNoise (
    uint16_t nb_samples)
```

Teste le bruit RMS du gyroscope pour différentes bandes passantes.

Parameters

<i>nb_samples</i>	Nombre de mesures pour calculer le RMS
-------------------	--

6.20.2.25 Variance()

```
void Variance (
    float data[][3],
    int N,
    float avg[3],
    float var[3])
```

Calcule la variance des N mesures gyroscopiques.

Parameters

<i>data</i>	Tableau [N][3]
<i>N</i>	Nombre d'échantillons
<i>avg</i>	Moyennes calculées pour chaque axe
<i>var</i>	Tableau de sortie [3] pour stocker les variances

6.21 Utils/Src/tools.c File Reference

Fonctions servant au tout début du projet.

```
#include "tools.h"
#include "string.h"
#include "stdio.h"
```

Macros

- `#define LED_G_PORT GPIOA`
- `#define LED_G_PIN GPIO_PIN_5`
- `#define BTN_PORT GPIOC`
- `#define BTN_PIN GPIO_PIN_13`
- `#define ACTIVE_LOW 0`
- `#define SAMPLE_RATE_HZ 100`

Functions

- static uint8_t **button_is_pressed** (void)
- void **setup** (void)
Initialisation UART + message d'accueil. Affiche un message de bienvenue sur le terminal série et prépare l'environnement de test.
- void **loop** (void)
Boucle principale : clignotement LED + affichage compteur UART.
- void **I2C_scan** (void)
Scan du bus I2C pour détecter les périphériques connectés. Balaye toutes les adresses possibles (0x00 à 0x7F) et affiche sur le terminal les adresses des périphériques répondant.
- void **test_LED** (void)
Test du fonctionnement des LEDs verte et rouge.
- int **SaveMagToCSV** (const char *filename, float *Bx, float *By, float *Bz, uint16_t N)
Enregistre les mesures du magnétomètre dans un fichier CSV.
- void **TestMagCSV** (void)

Variables

- UART_HandleTypeDef **hiuart1**
- I2C_HandleTypeDef **hi2c1**
- uint8_t **prompt** [15] = "\nSTM32G431 >> "
- uint8_t **message** [100]
- uint8_t **foundAddresses** [128]
- uint8_t **foundCount** = 0

6.21.1 Detailed Description

Fonctions servant au tout début du projet.

Author

Nicolas SIMOND

Date

Novembre 2025

6.21.2 Function Documentation

6.21.2.1 I2C_scan()

```
void I2C_scan (  
    void )
```

Scan du bus I2C pour détecter les périphériques connectés. Balaye toutes les adresses possibles (0x00 à 0x7F) et affiche sur le terminal les adresses des périphériques répondant.

Note

Utilise la fonction HAL_I2C_IsDeviceReady().

6.21.2.2 SaveMagToCSV()

```
int SaveMagToCSV (  
    const char * filename,  
    float * Bx,  
    float * By,  
    float * Bz,  
    uint16_t N)
```

Enregistre les mesures du magnétomètre dans un fichier CSV.

Parameters

<i>filename</i>	Nom du fichier CSV
<i>Bx</i>	Tableau des mesures X
<i>By</i>	Tableau des mesures Y
<i>Bz</i>	Tableau des mesures Z
<i>N</i>	Nombre de mesures

Returns

0 si OK, -1 si erreur

6.21.2.3 test_LED()

```
void test_LED (  
    void )
```

Test du fonctionnement des LEDs verte et rouge.

Allume la LED pendant 1 seconde puis l'éteint. Permet de vérifier la configuration GPIO et la fonction HAL_Delay.

